

# RISC-V SoC with Virtual Memory Support

Project 01 — RV32I Core with Simplified MMU, TLB and Page-Fault Handling

*Project Report by*

Atika Tufail

Zuha Sajjad

## Abstract

This project implements a synthesizable 32-bit RV32I multicycle processor integrated into a small System-on-Chip whose data memory accesses pass through a simplified virtual-memory subsystem. The subsystem contains a 4-entry fully-associative TLB, a 32-entry single-level page table, and a page-table controller that together translate 13-bit virtual addresses into 12-bit physical addresses. Valid mappings reach physical SRAM, while invalid mappings raise a sticky page-fault flag, suppress writes and return a defined fault pattern. An AXI4-Lite slave interface is used to program the page table and read fault, hit and miss status, and a UART/LED debug block exposes results on the FPGA. The design was verified with a directed, self-checking SystemVerilog testbench in which all checks pass, and a UVM environment and FPGA wrapper for the Arty A7-100T have been developed.

## 1. Introduction and Objectives

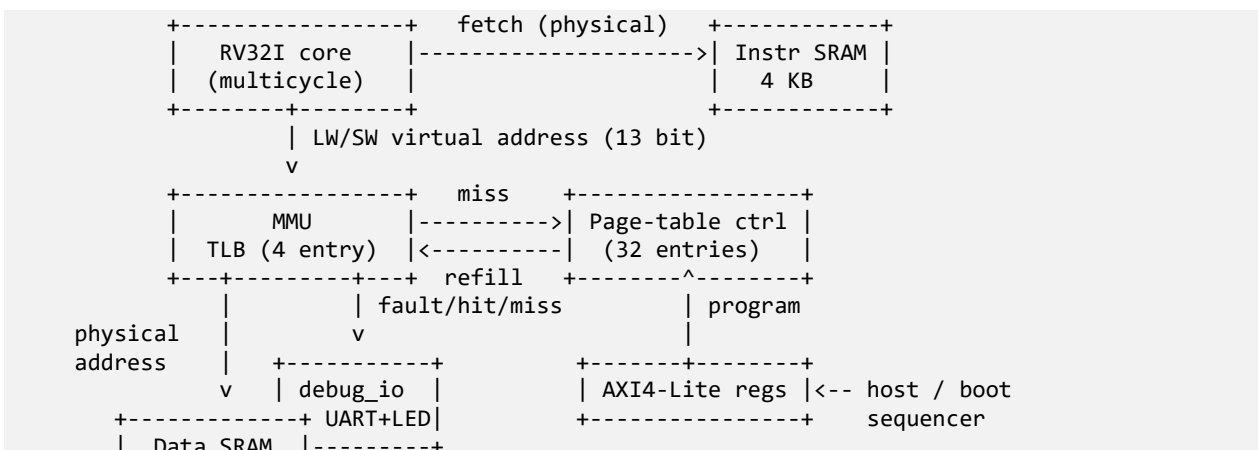
Virtual memory lets a processor use addresses that are independent of where data physically resides, and is the basis for protection and memory management in operating systems. This project builds a reduced version of that mechanism to demonstrate its fundamental concepts in hardware: address translation, page tables, TLB operation and page-fault handling. Full operating-system support, Linux capability and the privileged-mode MMU are explicitly out of scope.

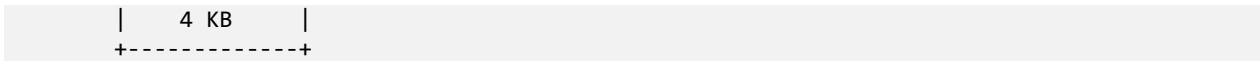
### 1.1 Objectives

- Design a synthesizable RV32I multicycle core with instruction memory, data memory and memory accesses routed through the MMU.
- Implement a simplified MMU with a virtual-to-physical translator, small TLB, page-table controller and page-fault/status registers.
- Provide an AXI4-Lite control interface for MMU configuration and status access.
- Provide a UART and LED debug interface to show successful and failed translations.
- Verify the design with a directed self-checking testbench and a UVM environment, and take it through FPGA implementation and physical design.

## 2. System Architecture

The SoC is composed of the following blocks. The core fetches instructions directly from physical instruction memory, while load and store addresses are treated as virtual addresses and sent to the MMU. The MMU consults the TLB in parallel with request issue; on a miss it reads the page table, refills the TLB and completes the access.





## 2.1 Module summary

| Module (file)                           | Function                                                                                                                                                  |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| rv32i_core<br>(rv32i_core.sv)           | Multicycle RV32I core: R-type and I-type ALU ops, LW/SW, six branches, JAL, JALR, Lui, AUIPC. Fetch is untranslated; LW/SW use the MMU.                   |
| instr_mem / data_mem<br>(memories.sv)   | 4 KB physical instruction SRAM and 4 KB physical data SRAM; combinational read, synchronous write behavioural models.                                     |
| tlb (tlb.sv)                            | 4-entry fully-associative TLB with parallel VPN compare and 2-bit round-robin replacement.                                                                |
| page_table_ctrl<br>(page_table_ctrl.sv) | 32-entry single-level page table (valid + 4-bit PPN), one-cycle lookup, programmable through the AXI registers.                                           |
| mmu (mmu.sv)                            | Combines TLB and page table; two-state FSM (IDLE / MISS) handling hit, miss-then-valid and miss-then-fault; drives physical data memory and fault status. |
| axi_lite_regs<br>(axi_lite_regs.sv)     | Simplified single-beat AXI4-Lite slave for page-table programming and status readback.                                                                    |
| debug_io / uart_tx                      | LED status stretching and UART transmitter (115200 baud) reporting MMU results.                                                                           |
| soc_top (soc_top.sv)                    | Top level integrating all blocks; fpga_top adds a boot sequencer for the Arty board.                                                                      |

## 3. Virtual Memory Design

### 3.1 Address translation parameters

| Parameter             | Value                        | Note                                         |
|-----------------------|------------------------------|----------------------------------------------|
| Virtual data address  | 13 bits (8 KB)               | VPN[12:8] + offset[7:0]                      |
| Physical data address | 12 bits (4 KB)               | PPN[11:8] + offset[7:0]                      |
| Page size             | 256 B                        | 8-bit offset                                 |
| Virtual pages         | 32 (5-bit VPN)               | Half of virtual space mappable at once       |
| Physical frames       | 16 (4-bit PPN)               | Enables misses and faults in short tests     |
| Page-table entry      | 5 bits                       | valid (1) + PPN (4)                          |
| TLB                   | 4 entries, fully associative | Entry = valid + VPN(5) + PPN(4); round-robin |

### 3.2 Memory map

| Region                        | Address range             | Notes                             |
|-------------------------------|---------------------------|-----------------------------------|
| Instruction memory (physical) | 0x0000_0000 – 0x0000_0FFF | 4 KB, addressed directly by PC    |
| Data space (virtual)          | 0x0000_0000 – 0x0000_1FFF | 8 KB as seen by LW/SW             |
| Data memory (physical)        | 0x0000_0000 – 0x0000_0FFF | 4 KB behind MMU                   |
| AXI4-Lite control/status      | 0x8000_0000 – 0x8000_00FF | Page-table programming and status |
| UART/GPIO debug               | 0x8000_0100 – 0x8000_01FF | Pass/fail indication              |

### 3.3 Translation flow

For every LW/SW the MMU splits the virtual address into VPN and offset and compares the VPN against all TLB entries. A hit forms the physical address {PPN, offset} and accesses data SRAM immediately. A miss starts a page-table lookup: if the entry is valid the TLB is refilled (round-robin) and the access completes; if it is invalid a page fault is raised.

### 3.4 Page-fault handling

Fault handling is deliberately exception-free. On an invalid mapping the write is suppressed, a read returns the sentinel 0xDEAD\_DEAD, and a sticky page\_fault flag is latched in the status register (readable over AXI4-Lite and shown on an LED). The core is not trapped and simply continues with its next instruction, which avoids trap and PC-redirect hardware that the specification excludes.

## 4. Bus and Control Interface

A simplified AXI4-Lite slave (single-beat, no pipelining) provides configuration and status. Offsets are relative to 0x8000\_0000.

| Offset | Register            | Description                                                       |
|--------|---------------------|-------------------------------------------------------------------|
| 0x00   | PAGE_TABLE_PROG (W) | Commits on write: bit0 = valid, bits[4:1] = PPN, bits[9:5] = VPN. |
| 0x04   | FAULT_STATUS (R/W)  | Read: bit0 = sticky page-fault flag. Any write clears the flag.   |
| 0x08   | TLB_DEBUG (R)       | {repl_ptr[1:0], entry_valid[3:0]}.                                |
| 0x0C   | HIT_COUNT (R)       | Free-running TLB hit counter.                                     |
| 0x10   | MISS_COUNT (R)      | Free-running TLB miss counter.                                    |

## 5. Verification

### 5.1 Directed self-checking testbench

The testbench (tb\_soc\_top.sv) programs the page table over AXI4-Lite while the core is held in reset, mapping VPN 0 to PPN 3 and leaving VPN 4 unmapped. It then releases the core to run an 11-instruction program, generated with a small Python RV32I assembler (asm.py, gen\_test\_prog.py), that stores and loads through the mapped page (first access a TLB miss, later ones hits) and through the unmapped page (page fault). The bench then checks registers, physical memory, and AXI status registers, and finally clears the sticky fault flag.

| Check                                    | Observed   | Result |
|------------------------------------------|------------|--------|
| x2 — LW through mapped page (expect 100) | 0x00000064 | PASS   |
| x4 — LW hit (expect 55)                  | 0x00000037 | PASS   |
| x8 — LW on unmapped page (fault pattern) | 0xDEADDEAD | PASS   |
| Physical memory word 192 (VA 0 data)     | 0x00000064 | PASS   |
| Physical memory word 193 (VA 4 data)     | 0x00000037 | PASS   |
| FAULT_STATUS sticky flag                 | 0x00000001 | PASS   |
| led_fault                                | 1          | PASS   |
| hit_count >= 3 / miss_count >= 3         | 3 / 3      | PASS   |

| Check                    | Observed   | Result |
|--------------------------|------------|--------|
| FAULT_STATUS after clear | 0x00000000 | PASS   |

Result: ALL CHECKS PASSED (full log in tb/sim\_output.txt). The simulation run also produces a VCD waveform showing hit, miss and fault behaviour.

## 5.2 UVM verification environment

A UVM environment wraps the same DUT and is organised as follows:

- AXI agent (driver, monitor, sequencer, sequence item) to program the page table and access status registers.
- MMU agent and monitor that observe TLB hit, miss and fault events as transaction items.
- Scoreboard that counts hits, misses and faults and flags a hit that does not complete, a fault without a preceding miss, or a fault that does not complete.
- Functional coverage: event kind (hit/miss/fault) x access type (read/write) and VPN region bins (low, mid, high).
- SystemVerilog assertions: BVALID and RVALID hold until accepted, fault requires a preceding miss, fault latches the status flag, hit completes the access, miss does not complete in the miss cycle.
- Sequences and tests: page\_table\_prog\_seq, fault\_seq, rand\_access\_seq with base\_test, fault\_test and rand\_test; reset is applied at the start of each test.

## 6. FPGA Implementation

The FPGA target is the Digilent Arty A7-100T (XC7A100T-CSG324-1) with a 100 MHz clock, built through an F4PGA flow (flow.json, Makefile). The fpga\_top wrapper contains a small boot sequencer that writes VPN 0 to PPN 3 into the page table through the AXI4-Lite interface, releases the core, and runs the same program as the simulation testbench. Results are shown on LEDs and over the USB-UART.

| Signal    | Pin | Function                                |
|-----------|-----|-----------------------------------------|
| clk       | E3  | 100 MHz system clock (10 ns constraint) |
| reset_n   | C2  | Active-low reset button                 |
| led_fault | H5  | Page fault indicator                    |
| led_hit   | J5  | TLB hit indicator                       |
| led_miss  | T9  | TLB miss indicator                      |
| led_boot  | T10 | Boot/run status                         |
| uart_txd  | D10 | UART TX to host (115200 baud)           |

## 7. Synthesis and Physical Design

The complete top-level design was taken through the physical-design flow. Figure 1 shows the final layout of the top-level design.

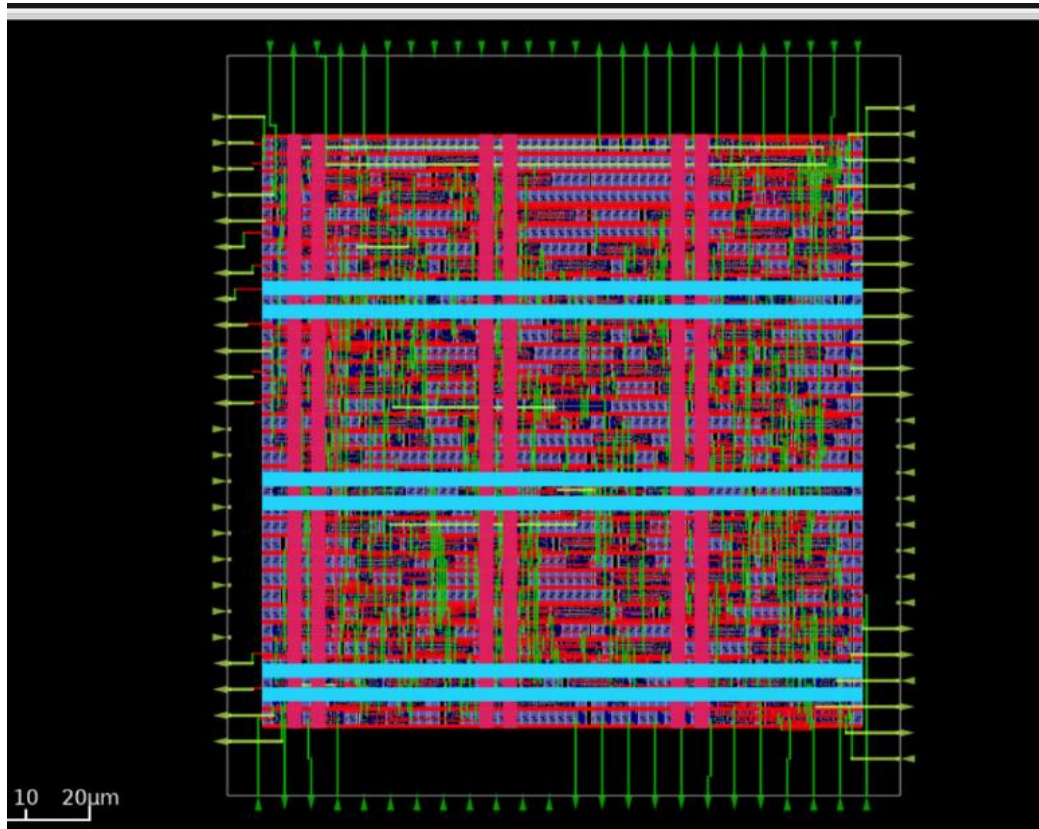


Figure 1: Final GDS layout of the complete top-level design (scale bar: 20  $\mu\text{m}$ ).

## 8. Design Limitations

The following simplifications are deliberate scoping decisions, not defects:

- No permission bits: every valid mapping is implicitly read and write.
- Single-level page table rather than the two-level Sv32 walk, because the address space is small.
- No ASID or multi-process isolation, and no TLB shutdown (single core).
- Instruction fetches are not translated, avoiding trap and PC-redirect hardware.
- Only LW and SW are implemented for memory access (no byte or halfword loads/stores).
- Page faults are reported through status and LEDs/UART rather than processor exceptions.

## 9. Conclusion

The project delivers a working RV32I SoC in which data accesses are translated through a TLB and page table, with page faults reported through status registers, an AXI4-Lite interface and a UART/LED debug path. The directed testbench demonstrates correct translation, TLB miss-then-hit behaviour, invalid-mapping fault reporting and status clearing.

## Appendix A: File Structure

```
rtl/  soc_pkg, tlb, page_table_ctrl, mmu, memories, rv32i_core,
      axi_lite_regs, debug_io, uart_tx, soc_top
tb/   tb_soc_top.sv, asm.py, gen_test_prog.py, test_prog.hex
uvm/  agents, env, scoreboard, coverage, assertions, sequences, tests
fpga/ fpga_top.sv, arty.xdc, flow.json, Makefile
docs/ architecture.md
```